day 18 | 251008 W

herein

We will work on root finding, which is a topic in chapter 6, but also a bit in chapter 5.

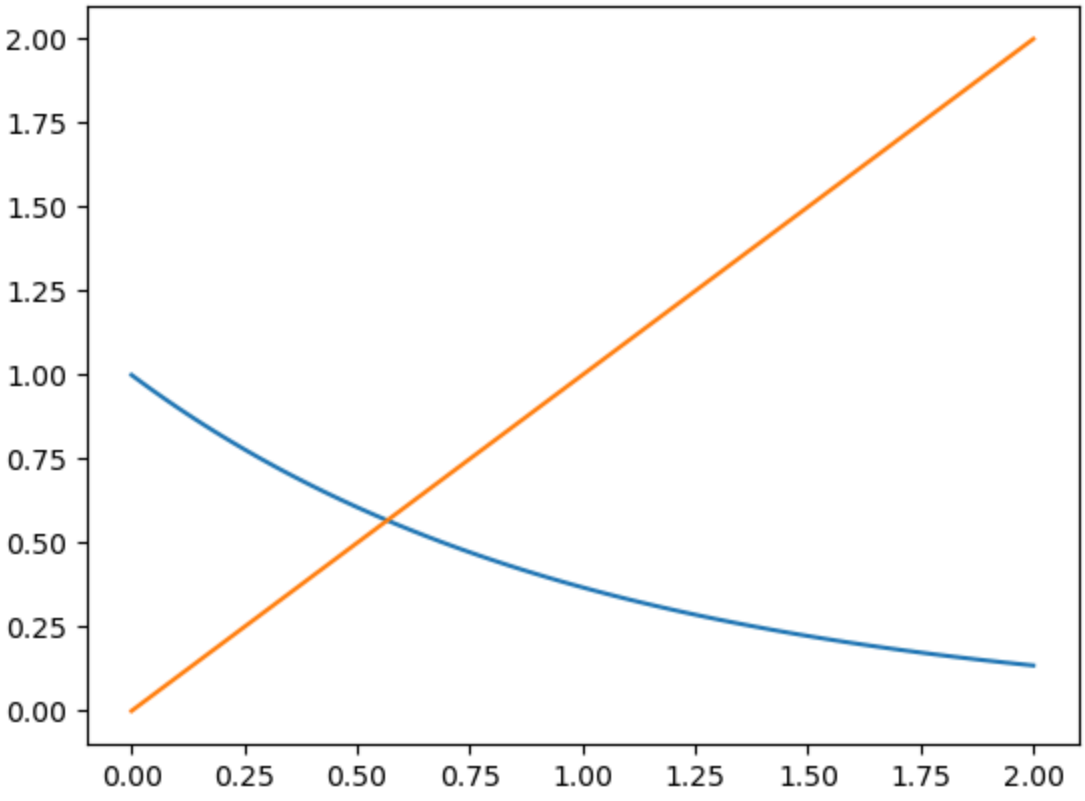
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

In [2]:
```
fig0, ax0 = plt.subplots()

x = np.linspace(0, 2, 100)
y = np.exp(-x)
z = x

ax0.plot(x, y)
ax0.plot(x, z)
```

Out[2]:
```
[<matplotlib.lines.Line2D at 0x70cc655b9010>]
```



In [9]:
```
df = pd.DataFrame({'dog': x, 'cat':y, 'parrot':z})
```

In [14]:
```
pd.set_option('display.max_rows', None)
```

In [15]:
```
df
```

Out[15]:

	dog	cat	parrot
0	0.000000	1.000000	0.000000
1	0.020202	0.980001	0.020202
2	0.040404	0.960401	0.040404
3	0.060606	0.941194	0.060606
4	0.080808	0.922371	0.080808
5	0.101010	0.903924	0.101010
6	0.121212	0.885846	0.121212
7	0.141414	0.868130	0.141414
8	0.161616	0.850768	0.161616
9	0.181818	0.833753	0.181818
10	0.202020	0.817078	0.202020
11	0.222222	0.800737	0.222222
12	0.242424	0.784723	0.242424
13	0.262626	0.769029	0.262626
14	0.282828	0.753649	0.282828
15	0.303030	0.738577	0.303030
16	0.323232	0.723806	0.323232
17	0.343434	0.709330	0.343434
18	0.363636	0.695144	0.363636
19	0.383838	0.681242	0.383838
20	0.404040	0.667617	0.404040
21	0.424242	0.654265	0.424242
22	0.444444	0.641180	0.444444
23	0.464646	0.628357	0.464646
24	0.484848	0.615790	0.484848
25	0.505051	0.603475	0.505051
26	0.525253	0.591406	0.525253
27	0.545455	0.579578	0.545455
28	0.565657	0.567987	0.565657

	dog	cat	parrot
29	0.585859	0.556628	0.585859
30	0.606061	0.545496	0.606061
31	0.626263	0.534586	0.626263
32	0.646465	0.523895	0.646465
33	0.666667	0.513417	0.666667
34	0.686869	0.503149	0.686869
35	0.707071	0.493086	0.707071
36	0.727273	0.483225	0.727273
37	0.747475	0.473561	0.747475
38	0.767677	0.464090	0.767677
39	0.787879	0.454809	0.787879
40	0.808081	0.445713	0.808081
41	0.828283	0.436799	0.828283
42	0.848485	0.428063	0.848485
43	0.868687	0.419502	0.868687
44	0.888889	0.411112	0.888889
45	0.909091	0.402890	0.909091
46	0.929293	0.394833	0.929293
47	0.949495	0.386936	0.949495
48	0.969697	0.379198	0.969697
49	0.989899	0.371614	0.989899
50	1.010101	0.364182	1.010101
51	1.030303	0.356899	1.030303
52	1.050505	0.349761	1.050505
53	1.070707	0.342766	1.070707
54	1.090909	0.335911	1.090909
55	1.111111	0.329193	1.111111
56	1.131313	0.322609	1.131313
57	1.151515	0.316157	1.151515

	dog	cat	parrot
58	1.171717	0.309834	1.171717
59	1.191919	0.303638	1.191919
60	1.212121	0.297565	1.212121
61	1.232323	0.291614	1.232323
62	1.252525	0.285782	1.252525
63	1.272727	0.280067	1.272727
64	1.292929	0.274466	1.292929
65	1.313131	0.268976	1.313131
66	1.333333	0.263597	1.333333
67	1.353535	0.258325	1.353535
68	1.373737	0.253159	1.373737
69	1.393939	0.248096	1.393939
70	1.414141	0.243134	1.414141
71	1.434343	0.238272	1.434343
72	1.454545	0.233506	1.454545
73	1.474747	0.228837	1.474747
74	1.494949	0.224260	1.494949
75	1.515152	0.219775	1.515152
76	1.535354	0.215380	1.535354
77	1.555556	0.211072	1.555556
78	1.575758	0.206851	1.575758
79	1.595960	0.202714	1.595960
80	1.616162	0.198660	1.616162
81	1.636364	0.194687	1.636364
82	1.656566	0.190793	1.656566
83	1.676768	0.186977	1.676768
84	1.696970	0.183238	1.696970
85	1.717172	0.179573	1.717172
86	1.737374	0.175982	1.737374

	dog	cat	parrot
87	1.757576	0.172462	1.757576
88	1.777778	0.169013	1.777778
89	1.797980	0.165633	1.797980
90	1.818182	0.162321	1.818182
91	1.838384	0.159074	1.838384
92	1.858586	0.155893	1.858586
93	1.878788	0.152775	1.878788
94	1.898990	0.149720	1.898990
95	1.919192	0.146725	1.919192
96	1.939394	0.143791	1.939394
97	1.959596	0.140915	1.959596
98	1.979798	0.138097	1.979798
99	2.000000	0.135335	2.000000

In [26]:
```
x = 1.1
for i in range(15):
    x = np.exp(1-x**2)
    print(x)
```

```
0.810584245970187
1.4091027864501258
0.3732261798634921
2.3648207320388734
0.010128752729784245
2.7180029697949326
0.0016823895519275452
2.718274134550982
0.0016799113235371871
2.7182741572011357
0.0016799111166751794
2.718274157203025
0.0016799111166579267
2.7182741572030253
0.0016799111166579221
```

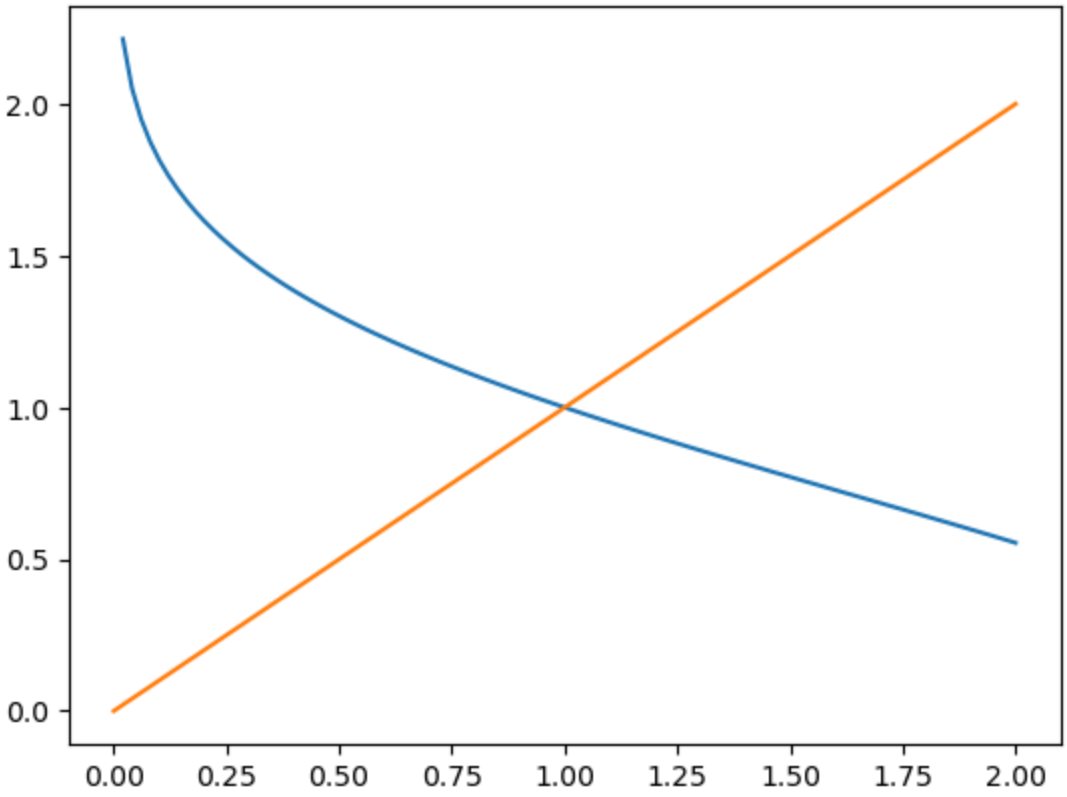
In [27]:
```
fig1, ax1 = plt.subplots()

x = np.linspace(0, 2, 100)
y = np.sqrt(1-np.log(x))
z = x

ax1.plot(x, y)
ax1.plot(x, z)
```

```
/tmp/ipykernel_124670/3263658484.py:4: RuntimeWarning: divide by zero encoun
tered in log
  y = np.sqrt(1-np.log(x))
```

Out[27]: [<matplotlib.lines.Line2D at 0x70cc647934d0>]



In [29]:
```
x = 0.1
for i in range(15):
    x = np.sqrt(1-np.log(x))
    print(x)
```

```
1.817301596597011
0.6345449064808987
1.2061704770622241
0.9014153062011224
1.0506137198769554
0.9750002622873489
1.012579645742329
0.993729752313783
1.0031400641243098
0.9984311972193877
1.0007847094356639
0.9996077222029717
1.0001961581400374
0.9999019257389472
1.0000490383329448
```

In [32]:
```
x = 2
error = 1
count = 0
while abs(error)>0.00001:
    x1 = np.sqrt(1-np.log(x))
```

```
    error = x1 - x
    x = x1
    count = count + 1
    if count > 100000:
        print('i couldnt find an answer so ignore this')
        break
print(x)
```

```
0.9999980668017239
```

In [34]:
```
x = 2
error = 1
count = 0
while abs(error)>0.00001:
    x1 = np.exp(1-x**2)
    error = x1 - x
    x = x1
    count = count + 1
    if count > 100000:
        print('i couldnt find an answer so ignore this')
        break
print(x)
```

```
i couldnt find an answer so ignore this
0.0016799111166579221
```

In [36]:
```
x = 2 # initial guess
error = 1 # initial error
count = 0 # counting index to see how many steps
while abs(error)>0.00001:
    x1 = np.exp(-x)
    error = x1 - x
    x = x1
    count = count + 1
    if count > 100000:
        print('i couldnt find an answer so ignore this')
        break
print(x)
print(count)
```

```
0.5671465647812954
22
```

In [ ]:
```

```